

PROJECT REPORT

GUESS THE NUMBER GAME

A Python Command-Line Project

Student Name	Hansika Singh
Registration Number	26MIP10001
Project	Guess the Number Game
Programming Language	Python

Submitted as a Python Programming Project

1. Introduction

The Guess the Number Game is a simple interactive Python project in which the computer randomly selects a number from 1 to 100 and the player attempts to guess it. After every valid guess, the program provides a useful hint. The project demonstrates basic Python concepts such as functions, loops, conditional statements, exception handling, input validation, random number generation, and formatted output.

2. Objective

- To develop a simple command-line game using Python.
- To understand the use of the random module.
- To practice while loops and conditional statements.
- To handle invalid user input using exception handling.
- To count and display the number of attempts taken by the player.

3. Problem Statement

Design and implement a Python program that generates a random number between 1 and 100. The user repeatedly enters guesses until the correct number is found. The program should indicate whether the guess is too high, too low, or very close, and should count valid attempts.

4. Requirements

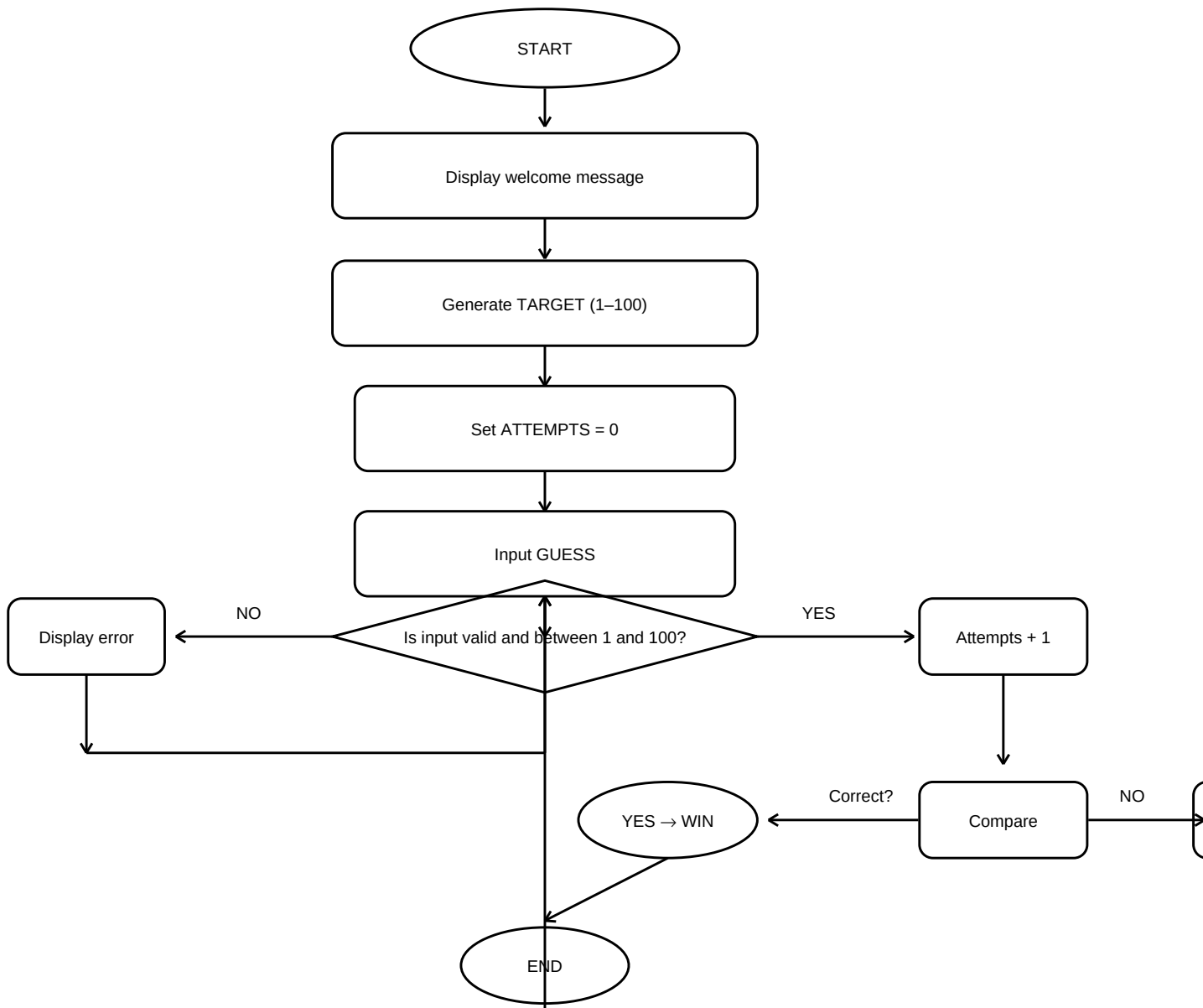
Hardware	A computer/laptop with basic processing capability
Software	Python 3.x
Library	random (built-in Python module)
Interface	Command Line / Terminal

5. Algorithm

1. Start the program.
2. Display the welcome message and explain that the target is between 1 and 100.
3. Generate a random target number between 1 and 100.
4. Initialize the attempt counter to 0.
5. Ask the user to enter a guess.
6. Check whether the input is a valid integer. If not, display an error message and ask again.
7. Check whether the guess is between 1 and 100. If not, display a range message and ask again.
8. Increase the number of attempts by 1.
9. Calculate the absolute difference between the target and the guess.
10. If the guess equals the target, display the success message and number of attempts, then stop.
11. If the difference is 5 or less, display a 'Very close' hint.
12. If the guess is less than the target, display 'Too low'. Otherwise, display 'Too high'.
13. Repeat until the user guesses the correct number.
14. End the program.

6. Flowchart

The flowchart represents the main logic of the Guess the Number Game.



7. Pseudocode

```
BEGIN
  Display welcome message
  Generate TARGET randomly from 1 to 100
  Set ATTEMPTS = 0

  REPEAT
    Read GUESS

    IF input is not a valid number
      Display "Invalid input"
      Continue
    END IF

    IF GUESS is outside 1 to 100
      Display "Please guess a number between 1 and 100"
      Continue
    END IF

    Increase ATTEMPTS by 1
    DIFFERENCE = absolute value of TARGET - GUESS

    IF GUESS = TARGET
      Display congratulations and ATTEMPTS
      Stop
    ELSE IF DIFFERENCE <= 5
      Display "Very close"
    ELSE IF GUESS < TARGET
      Display "Too low"
    ELSE
      Display "Too high"
    END IF
  UNTIL correct guess

END
```

8. Python Source Code

```
import random

def play_game():
    print("Welcome to the Guess the Number Game!")
    print("I'm thinking of a number between 1 and 100. Can you guess it?")

    target = random.randint(1, 100)
    attempts = 0

    while True:
        try:
            guess = int(input("Enter your guess: "))
        except ValueError:
            print("Invalid input. Please enter a number between 1 and 100.")
            continue
        except EOFError:
            print("\nGame ended. Goodbye!")
            break

        if guess < 1 or guess > 100:
            print("Please guess a number between 1 and 100.")
            continue

        attempts += 1
        difference = abs(target - guess)

        if guess == target:
            print(f"Congratulations! You guessed it in {attempts} attempts.")
            break
        elif difference <= 5:
            print("Very close! Keep trying.")
        elif guess < target:
            print("Too low! Try a higher number.")
        else:
            print("Too high! Try a lower number.")

if __name__ == "__main__":
    play_game()
```

9. Explanation of the Program

random module: The random module generates the target number.

play_game(): This function contains the complete game logic.

random.randint(1, 100): Generates an integer from 1 through 100, inclusive.

while True: Keeps the game running until the correct number is guessed or the input stream ends.

try-except: Handles non-numeric input so the program does not crash.

attempts: Counts the number of valid guesses.

difference: Stores the absolute distance between the target and the guess.

if / elif / else: Determines the appropriate response to each guess.

__name__ check: Runs play_game() when the file is executed directly.

10. Sample Output

```
Welcome to the Guess the Number Game!
I'm thinking of a number between 1 and 100. Can you guess it?
Enter your guess: 40
Too low! Try a higher number.
Enter your guess: 70
Very close! Keep trying.
Enter your guess: 74
Congratulations! You guessed it in 3 attempts.
```

Note: The target number and output vary because the number is generated randomly.

11. Testing

Test Case	Input	Expected Result
Correct guess	Target number	Congratulations message and attempt count
Too low	Less than target	Too low hint
Too high	Greater than target	Too high hint
Very close	Difference ≤ 5	Very close hint
Out of range	0 or 101	Range warning
Invalid input	abc	Invalid input warning

12. Advantages

- Simple and easy to understand.
- Provides immediate feedback after each guess.
- Handles invalid input without terminating unexpectedly.
- Demonstrates important Python programming concepts.

- Can be extended with additional game features.

13. Limitations

- The game supports only one player at a time.
- The target range is fixed at 1 to 100.
- There is no graphical user interface.
- The score is not saved after the program ends.

14. Future Enhancements

- Add difficulty levels.
- Add a maximum number of attempts.
- Add a scoring system.
- Allow the player to choose the number range.
- Create a graphical user interface.
- Store high scores.

15. Conclusion

The Guess the Number Game successfully demonstrates how Python can be used to create an interactive command-line application. The project combines random number generation, loops, conditional logic, functions, input validation, exception handling, and formatted output. It provides a practical example of applying fundamental programming concepts to a small project.

16. References

- Python 3 programming concepts: functions, loops, conditions, and exception handling.
- Python standard library concept: random number generation.